P0727R0
Jens Maurer <jens.maurer@gmx.net>
2017-07-10

P0727R0: Core Issue 1299: Temporary objects vs temporary expressions

This paper presents the proposed wording to resolve core issue 1299. The drafting below also addresses core issues 943 and 1076.

Proposed wording

Change in 8 expr paragraph 12:

> ... [Note: If the expression is an lvalue of class type, it must have a volatile copy constructor to initialize the temporary **object** that is the result object of the lvalue-to-rvalue conversion. -- end note] ...

Change in 15.2 class.temporary paragraph 6:

> The third context is when a reference is bound to a temporary **object**. [Footnote: The same rules apply to initialization of an initializer_list object (8.6.4) with its underlying temporary array.] The temporary **object** to which the reference is bound or the temporary **object** that is the complete object of a subobject to which the reference is bound persists for the lifetime of the reference **if the glvalue to which the reference is bound was obtained through one of the following:**
>
> - **a temporary materialization conversion (7.4 [conv.rval]),**
> - **(*expression*), where *expression* is one of these expressions,**
> - **subscripting (8.2.1 [expr.sub]) of an array operand, where that operand is one of these expressions,**
> - **a class member access (8.2.5 [expr.ref]) using the `.` operator where the left operand is one of these expressions and the right operand designates a non-static data member of non-reference type,**
> - **a pointer-to-member operation (8.5 [expr.mptr.oper]) using the `.*` operator where the left operand is one of these expressions and the right operand is a pointer to data member of non-reference type,**
> - **a `const_cast` (8.2.11 [expr.const.cast]), `static_cast` (8.2.9 [expr.static.cast]), `dynamic_cast` (8.2.7 [expr.dynamic.cast]), or `reinterpret_cast` (8.2.10 [expr.reinterpret.cast]) converting, without a user-defined conversion, a glvalue operand that is one of these expressions to a glvalue that refers to the object designated by the operand, or to its complete object or a subobject thereof,**
> - **a conditional expression (8.16 [expr.cond]) that is a glvalue where the second or third operands are one of these expressions,**
> - **a comma expression (8.19 [expr.comma]) where the right operand is one of these expressions.**
>
> **[Example:**
>
> ```
> template<typename T> using id = T;
>
> int&& a = id<int[3]>{1, 2, 3}[i];           // temporary array has same lifetime as a
> const int& b = static_cast<const int&>(0);  // temporary int has same lifetime as b
> ```

```
int&& c = cond ? id<int[3]>{1, 2, 3}[i] : static_cast<int&&>(0);
          // exactly one of the two temporaries is lifetime-extended
```

] [Note: An explicit type conversion (8.2.3 [expr.type.conv] and 8.4 [expr.cast]) is interpreted as a sequence of elementary casts, covered above. [Example:

```
  const int& x = (const int&)1;   // temporary for value 1 has same lifetime as x
```

-- end example] -- end note] [Note: If a temporary object has a reference member initialized by another temporary object, lifetime extension applies recursively to such a member's initializer. [Example:

```
struct S {
  const int& m;
};
const S& s = S{1};   // both "S" and "int" temporaries have lifetime of s
```

-- end example] -- end note] ~~except:~~

The exceptions to this lifetime rule are:

- A temporary object bound to a reference parameter in a function call (8.2.2) persists until the completion of the full-expression containing the call.
- The lifetime of a temporary bound to the returned value in a function return statement (9.6.3) is not extended; the temporary is destroyed at the end of the full-expression in the return statement.
- A temporary bound to a reference in a new-initializer (8.3.4) persists until the completion of the full-expression containing the new-initializer. [Example: ...]

Change in 16.3.1.4 over.match.copy paragraph 1:

- ...
- When the type of the initializer expression is a class type "cv S", the non-explicit conversion functions of S and its base classes are considered. When initializing a temporary **object (12.2 [class.temporary])** to be bound to the first parameter of a constructor that takes a reference to possibly cv-qualified T as its first argument, called with a single argument in the context of direct-initialization of an object of type "cv2 T", explicit conversion functions are also considered. ...

Change in 20.5.4.9 res.on.arguments paragraph 1:

- ...
- .. [Note: If a program casts an lvalue to an xvalue while passing that lvalue to a library function (e.g. by calling the function with the argument `move(x)`), the program is effectively asking that function to treat that lvalue as a temporary **object**. The implementation is free to optimize away aliasing checks which might be needed if the argument was an lvalue. -- end note]

Change in 23.11.2.3.3 util.smartptr.weak.assign paragraph 2:

Remarks: The implementation may meet the effects (and the implied guarantees) via different means, without creating a temporary **object**.

Change in 29.7.2.1 template.valarray.overview paragraph 1:

... The illusion of higher dimensionality may be produced by the familiar idiom of computed indices, together with the powerful subsetting capabilities provided by the generalized subscript operators. [Footnote: The intent is to specify an array template that has the minimum functionality necessary to

address aliasing ambiguities and the proliferation of ~~temporaries~~ **temporary objects**. Thus, the valarray template is neither a matrix class nor a field class. However, it is a very useful building block for designing such classes.]

Change in C.2.16 diff.cpp03.input.output:

- ...
- initializing a const bool& which would bind to a temporary **object**.

[...] a const_cast, static_cast, dynamic_cast, or reinterpret_cast converting, without a user-defined conversion, a glvalue operand that is one of these expressions to a glvalue that refers to the object designated by the operand, or to its complete object or a subobject thereof, [...] For (c) above, I suggest: [...] if the glvalue to which the reference is bound was obtained through one of the following: [...]